



Informe técnico de avances AWA05_0003

Sistema de telemetría, análisis estadístico B01, comparativo de calidad de agua y arquitectura informática del nodo

Nodo Raspberry Pi · AWA05 · Santa Catarina Palopó · Lago de Atitlán

Campo	Valor técnico
Repositorio evaluado	awa05-telemetry-main
Informe base de referencia	Informe avances AWA05 (1).pdf
Bloque estadístico principal	B01 · Bloque 01 · datos disponibles actuales
Periodo de nivel B01	2026-04-23 12:29:11 a 2026-05-06 09:27:12
Periodo climático B01	2026-04-23 12:30:20 a 2026-04-29 06:13:50
Registros crudos actuales de nivel	1016 filas en data/raw/nivel_raw.csv
Registros crudos actuales de clima	496 filas en data/raw/clima_raw.csv
Calidad de agua	Lote 1 · muestra 2026-05-03 · laboratorio Atitlab · referencia COGUANOR NTG 290001
Acceso remoto observado	Tailscale IPv4 100.110.98.27

Dictamen de avance: el repositorio documenta una transición desde prototipo de adquisición hacia nodo embebido operativo. La unidad actual captura nivel por ultrasonido, recibe clima por HTTP, procesa datos para tablero, publica archivos al repositorio remoto, expone métricas de salud de la Raspberry Pi e incorpora salida analítica estática para estadística y calidad de agua

1. Alcance del documento actualizado

El presente informe actualiza el documento técnico de avances del sistema AWA05. La actualización incorpora tres capas nuevas: ampliación informática del nodo Raspberry Pi, análisis estadístico formal del bloque B01 y comparativo de calidad de agua del Lote 1. El texto se restringe a arquitectura, datos, implementación, control de calidad, inferencia estadística y criterios de operación. No se incorpora presupuesto ni ejecución financiera

La sección informática se construye a partir de scripts Python, archivos de configuración, datasets crudos, JSON procesados y estructura del repositorio. La sección estadística se deriva del archivo statistical_analysis.json y del módulo analisis-estadistico.html. La sección de calidad de agua se deriva de la matriz de parámetros incorporada en comparativo-calidad-agua.html y del archivo water_quality_data.json reconstruido desde dicho módulo

La interpretación mantiene separación estricta entre operación de nodo, rendimiento hídrico y calidad sanitaria. La operación del nodo prueba continuidad informática; la estadística B01 caracteriza producción bajo condiciones climáticas observadas; la calidad de agua describe un lote de laboratorio. Ninguna de esas capas sustituye a las otras

Capa	Fuente primaria	Resultado incorporado
Arquitectura informática	scripts/*.py, config/*.json, data/processed/dashboard_data.json	Diagrama lógico, responsabilidades por proceso, Tailscale, GitHub, scheduler, watchdog y salud del nodo
Telemetría cruda	data/raw/nivel_raw.csv, data/raw/clima_raw.csv	Volumen, distancia, clima reciente, periodicidad y consistencia temporal
Estadística B01	analisis-estadistico.html, statistical_analysis.json	Calidad de datos, supuestos, correlaciones, ANOVA, modelos, bootstrap y limitaciones
Calidad de agua	comparativo-calidad-agua.html, water_quality_data.json	Lote 1, comparación con fuentes comerciales, matriz de cumplimiento y cautelas de comparabilidad

2. Estado técnico actualizado del sistema

El sistema ya no debe describirse como repositorio preparatorio sin datos. El estado actual incluye 1.016 registros de nivel, 496 registros meteorológicos recientes, un JSON operativo de tablero, un bloque estadístico B01 generado con 978 registros de agua y 8.076 registros climáticos válidos, y una matriz de calidad de agua con 30 parámetros y 10 fuentes comparativas

Componente	Estado observado	Evidencia numérica o de código
Nodo Raspberry Pi	Operativo como nodo de borde	CPU 35.5 °C, RAM 10.8 %, disco 10.4 %, uptime 8h 39m, throttling 0x0
Nivel JSN-SR04T	Activo en dataset crudo	1016 filas; volumen final 10,04 L; distancia final 53,3 cm
Clima WS-2000	Activo en dataset crudo	496 filas; periodo 2026-05-11 17:44:18 a 2026-05-12 02:12:38
Dashboard JSON	Implementado	Actualizado 2026-05-12 01:46:52; contiene kpis, sistema y series
Scheduler	Implementado con ciclos residentes	Lectura, KPIs, subida a GitHub y watchdog término programados mediante schedule
Tailscale	Integrado como métrica operativa	tailscale_ip reportado: 100.110.98.27
GitHub Pages	Implementado	HTML estático para dashboard, análisis estadístico y calidad de agua
Calidad de agua	Lote 1 cargado	28 valores AWA05 reportados dentro de una matriz de 30 parámetros

2.1 Diferencias técnicas frente al informe anterior

Elemento	Informe anterior	Estado actual
----------	------------------	---------------



Elemento	Informe anterior	Estado actual
Geometría del recipiente	ALTURA_TOTAL=50,0 cm; ALTURA_MAX_AGUA=42,5 cm; RADIO=13,5 cm	ALTURA_TOTAL=72,5 cm; ALTURA_MAX_AGUA=38,3 cm; RADIO=12,9 cm; volumen máximo aproximado 20,0 L
Lectura ultrasónica	Una lectura defensiva con timeout	Mediana de 11 muestras, pausa de 0,06 s, timeout de 0,04 s y validación de rango
Frecuencia operativa	Lectura cada 60 min y subida 12:00/00:00	scheduler.py programa lectura cada 15 min, KPIs cada 15 min y watchdog cada 5 min
Persistencia	nivel_raw.csv y clima_raw.csv pendientes de poblar	nivel_raw.csv contiene 1016 filas y clima_raw.csv contiene 496 filas recientes
Procesamiento	CSV crudo y subida remota	process_data.py genera dashboard_data.json con KPIs de sistema, clima y nivel
Acceso remoto	SSH y mDNS	SSH/mDNS más observación de IP Tailscale en el tablero operativo
Análisis	No incorporado	B01 estadístico y Lote 1 de calidad de agua incorporados como HTML analíticos

3. Arquitectura informática de la Raspberry Pi como nodo de telemetría

La Raspberry Pi opera como nodo de borde con cinco responsabilidades simultáneas: adquisición local por GPIO, ingesta meteorológica HTTP, normalización y persistencia de datos, cálculo de KPIs operativos y publicación remota. El nodo no depende de ejecución manual para la ruta ordinaria: scheduler.py gobierna los ciclos internos y systemd debe mantener el proceso residente en arranque

Capa física

JSN-SR04T en GPIO17/GPIO18 + estación WS-2000 enviando HTTP

Capa de adquisición

read_distance.py mide distancia por tiempo de vuelo y read_ws2000.py recibe variables por /data

Capa de persistencia

main.py y utils.py escriben data/raw/nivel_raw.csv y data/raw/clima_raw.csv

Capa de procesamiento

process_data.py limpia outliers, convierte unidades, calcula KPIs y genera dashboard_data.json

Capa de operación

scheduler.py ejecuta lectura, procesamiento, publicación y watchdog térmico

Capa de publicación

upload_github.py sincroniza CSV y JSON con GitHub mediante token externo

Capa de acceso remoto

SSH local, mDNS y Tailscale IPv4 para administración privada fuera de la LAN

3.1 Inventario modular del repositorio

Archivo	Tamaño
README.md	92 bytes
Raspberry_Pi-Logo.wine.png	51.470 bytes
aala_logo.png	5.183 bytes

Archivo	Tamaño
analisis-estadistico.html	222.439 bytes
comparativo-calidad-agua.html	131.994 bytes
config/calibration.json	127 bytes
config/settings.json	334 bytes
dashboard.html	172.021 bytes
data/processed/dashboard_data.json	58.867 bytes
data/processed/statistical_analysis.json	139.161 bytes
data/raw/clima_raw.csv	30.619 bytes
data/raw/nivel_raw.csv	30.882 bytes
data/raw/peso_raw.csv	181 bytes
index.html	51.926 bytes
scripts/main.py	732 bytes
scripts/process_data.py	7.740 bytes
scripts/read_distance.py	3.085 bytes
scripts/read_weight.py	611 bytes
scripts/read_ws2000.py	1.401 bytes
scripts/scheduler.py	1.962 bytes
scripts/upload_github.py	2.734 bytes
scripts/utills.py	862 bytes

3.2 Driver de nivel ultrasónico

El driver `read_distance.py` implementa un flujo de bajo nivel sobre RPi.GPIO. El pulso TRIG se sostiene por 10 microsegundos, el pin ECHO se mide mediante dos ventanas con deadline de 0,04 s y la distancia se calcula con velocidad del sonido de 34.300 cm/s. La arquitectura vigente mejora la lectura anterior porque ya no depende de una muestra única; usa 11 intentos, descarta mediciones sin eco y retorna la mediana. Esa decisión reduce sensibilidad frente a rebotes, oleaje y lecturas espurias

Constante	Valor	Lectura técnica
TRIG/ECHO	GPIO17 / GPIO18	Interfaz BCM directa para control del JSN-SR04T
ALTURA_TOTAL	72,5 cm	Distancia vertical máxima del montaje al fondo físico
ALTURA_MAX_AGUA	38,3 cm	Altura útil saturada para cálculo de volumen
RADIO	12,9 cm	Radio interno usado para área basal
NUM_MUESTRAS	11	Número impar que permite mediana exacta
TIMEOUT_ECHO	0,04 s	Cierre de rutina ante ausencia de flanco
Volumen estimado máximo	20,02 L	Coherente con recipiente de escala 20 L

Bloque de código 1. Driver completo JSN-SR04T con mediana, timeout y conversión geométrica

```
# Bloque 1/4
import RPi.GPIO as GPIO
import time
import logging
```



```
TRIG = 17
ECHO = 18
```

```
ALTURA_TOTAL = 72.5
ALTURA_MAX_AGUA = 38.3
RADIO = 12.9
AREA_BASE = 3.14159265 * (RADIO ** 2)
DISTANCIA_MIN = ALTURA_TOTAL - ALTURA_MAX_AGUA
DISTANCIA_MAX = ALTURA_TOTAL
```

```
NUM_MUESTRAS = 11
PAUSA_MUESTRAS = 0.06
TIMEOUT_ECHO = 0.04
log = logging.getLogger("read_distance")
if not log.handlers:
    handler = logging.StreamHandler()
    handler.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(message)s", datefmt="%Y-%m-%d
    %H:%M:%S"))
    log.addHandler(handler)
    log.setLevel(logging.INFO)

def setup():
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)
    GPIO.setup(TRIG, GPIO.OUT)
    GPIO.setup(ECHO, GPIO.IN)
    GPIO.output(TRIG, False)
    time.sleep(0.5)
```

Bloque 2/4

```
def _medir_una_vez():
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)
    deadline = time.time() + TIMEOUT_ECHO
    while GPIO.input(ECHO) == 0:
        inicio = time.time()
        if inicio > deadline:
            log.debug("Timeout flanco ascendente")
            return None
    deadline = time.time() + TIMEOUT_ECHO
    while GPIO.input(ECHO) == 1:
        fin = time.time()
        if fin > deadline:
            log.debug("Timeout flanco descendente")
            return None
    return round((fin - inicio) * 34300 / 2, 1)

def medir_distancia_promedio(n=NUM_MUESTRAS):
    muestras = []
    for i in range(n):
        v = _medir_una_vez()
        if v is not None:
            muestras.append(v)
            log.debug(f"Muestra {i+1}/{n}: {v} cm")
        else:
            log.debug(f"Muestra {i+1}/{n}: descartada")
        if i < n - 1:
            time.sleep(PAUSA_MUESTRAS)
    if not muestras:
        return None
```

Bloque 3/4

```

muestras.sort()
mid = len(muestras) // 2
if len(muestras) % 2 == 0:
    return round((muestras[mid-1] + muestras[mid]) / 2, 1)
return round(muestras[mid], 1)

def distancia_a_volumen(distancia_cm):
    altura = max(0.0, min(ALTURA_TOTAL - distancia_cm, ALTURA_MAX_AGUA))
    return round(altura, 1), round(AREA_BASE * altura / 1000.0, 2)

def leer_nivel():
    setup()
    distancia = medir_distancia_promedio()
    GPIO.cleanup()
    if distancia is None:
        log.warning("Sin respuesta JSN-SR04T — verificar TRIG(GPIO17)/ECHO(GPIO18) y 5V.")
        return None, None
    if distancia < DISTANCIA_MIN:
        log.warning(f"Distancia {distancia} cm bajo minimo ({DISTANCIA_MIN} cm) — recipiente lleno.")
    elif distancia > DISTANCIA_MAX:
        log.warning(f"Distancia {distancia} cm sobre maximo ({DISTANCIA_MAX} cm) — fuera de rango.")
    altura, volumen = distancia_a_volumen(distancia)
    log.info(f"Distancia: {distancia} cm | Agua: {altura} cm | Volumen: {volumen} L")
    return distancia, volumen

if __name__ == "__main__":
    logging.basicConfig(level=logging.WARNING)
    print("=== Prueba JSN-SR04T — AWA05 ===")
    dist, vol = leer_nivel()
    if dist is not None:
        print(f"Resultado: {dist} cm | {vol} L")
    else:

```

Bloque 4/4

```
print("Sin lectura. Revisar conexion del sensor.")
```

3.3 Orquestador de lectura y política de descarte

main.py preserva integridad de datos crudos al escribir únicamente cuando leer_nivel() devuelve una distancia interpretable. Si el driver retorna None, el orquestador descarta la muestra y evita introducir filas vacías o no físicas en nivel_raw.csv. Esta lógica es correcta para un nodo autónomo: la falla de una lectura no debe contaminar el dataset ni detener el proceso residente

Bloque de código 2. Orquestación síncrona de una lectura válida hacia nivel_raw.csv

```

import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from scripts.utils import timestamp_ahora, guardar_csv
from scripts.read_distance import leer_nivel

RUTA_RAW = "data/raw/nivel_raw.csv"

def tomar_lectura():
    distancia, volumen = leer_nivel()
    if distancia is None:
        print("[SKIP] Lectura descartada, sin respuesta del sensor.")
        return
    fila = {
        "timestamp": timestamp_ahora(),
        "distancia_cm": distancia,
        "volumen_litros": volumen
    }
    guardar_csv(RUTA_RAW, fila)

```

```
print(f"[{fila['timestamp']}] Nivel: {distancia} cm | Volumen: {volumen} L")
```

```
if __name__ == "__main__":
    print("Sistema AWA05 iniciado.")
    tomar_lectura()
```

3.4 Ingesta meteorológica WS-2000 por HTTP

read_ws2000.py implementa un servidor Flask de baja complejidad. El endpoint /data acepta GET o POST, normaliza variables de la estación y persiste cada solicitud válida en clima_raw.csv. La decisión de mapear parámetros externos a columnas internas reduce acoplamiento entre el protocolo de la estación y el formato analítico del repositorio

Parámetro recibido	Columna persistida	Función
tempf	temp_exterior	Temperatura exterior en °F para conversión posterior
humidity	humedad	Humedad relativa porcentual
dewptf	punto_rocio	Punto de rocío en °F
baromin	presion	Presión barométrica en inHg
windspeedmph	viento_vel	Velocidad del viento en mph
winddir	viento_dir	Dirección de viento en grados
rainin	lluvia_hora	Lluvia horaria en pulgadas/h
solarradiation	radiacion_solar	Radiación solar en W/m²
UV	uv	Índice ultravioleta

Bloque de código 3. Servidor Flask de ingesta WS-2000

```
# Bloque 1/2
import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from flask import Flask, request
from scripts.utils import timestamp_ahora, guardar_csv

app = Flask(__name__)

RUTA_CLIMA = "data/raw/clima_raw.csv"

@app.route("/data", methods=["POST", "GET"])
def recibir_datos():
    datos = request.args if request.method == "GET" else request.form
    if not datos:
        return "Sin datos", 400

    fila = {
        "timestamp": timestamp_ahora(),
        "temp_exterior": datos.get("tempf", "N/A"),
        "humedad": datos.get("humidity", "N/A"),
        "punto_rocio": datos.get("dewptf", "N/A"),
        "presion": datos.get("baromin", "N/A"),
        "viento_vel": datos.get("windspeedmph", "N/A"),
        "viento_dir": datos.get("winddir", "N/A"),
        "lluvia_hora": datos.get("rainin", "N/A"),
        "radiacion_solar": datos.get("solarradiation", "N/A"),
        "uv": datos.get("UV", "N/A"),
    }

    guardar_csv(RUTA_CLIMA, fila)
```

```
print(f'[WS2000] Datos recibidos: temp={fila["temp_exterior"]} humedad={fila["humedad"]}')
return "OK", 200
```

```
@app.route("/", methods=["GET"])
def estado():
    return "Servidor AWA05 activo", 200
```

Bloque 2/2

```
def iniciar_servidor():
    app.run(host="0.0.0.0", port=7777, debug=False)

if __name__ == "__main__":
    iniciar_servidor()
```

3.5 Procesamiento, limpieza de outliers y generación del tablero

process_data.py constituye la capa de transformación entre datos crudos y tablero. El módulo convierte unidades meteorológicas, recorta el volumen a las últimas 9.999 observaciones, filtra saltos positivos mayores a 6 L y corrige descensos menores a 5 L como outliers plausiblemente asociados a viento u oscilación. Este criterio no elimina el dato crudo; genera una vista filtrada para visualización y KPIs

La misma rutina concentra salud del nodo. El sistema consulta vcgencmd, /proc/meminfo, statvfs, /proc/uptime, iwconfig, /proc/loadavg y Tailscale. La salida se almacena en data/processed/dashboard_data.json, de forma que el tablero no solo describe agua y clima, sino también estado computacional de la Pi

Bloque de código 4. Conversión de unidades, lectura CSV y extracción inicial de salud del sistema

Bloque 1/3

```
import csv
import json
import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from scripts.utils import timestamp_ahora

RUTA_NIVEL = "data/raw/nivel_raw.csv"
RUTA_CLIMA = "data/raw/clima_raw.csv"
RUTA_JSON = "data/processed/dashboard_data.json"

def f_a_c(f):
    try: return round((float(f) - 32) * 5 / 9, 1)
    except: return None

def mph_a_kmh(v):
    try: return round(float(v) * 1.60934, 1)
    except: return None

def leer_csv(ruta):
    if not os.path.exists(ruta):
        return []
    with open(ruta, "r") as f:
        return list(csv.DictReader(f))

def estado_sistema():
    import subprocess, re
    try:
        temp = float(re.search(r"temp=(\d.\d)+",
            subprocess.check_output(["vcgencmd", "measure_temp"]).decode()).group(1))
    except: temp = None
    try:
        volt = float(re.search(r"volt=(\d.\d)+",
```

Bloque 2/3

```

subprocess.check_output(["vgencmd","measure volts"]).decode()).group(1))
except: volt = None
try:
    mem = open("/proc/meminfo").read()
    total = int(re.search(r"MemTotal:\s+(\d+)", mem).group(1))
    available = int(re.search(r"MemAvailable:\s+(\d+)", mem).group(1))
    ram_pct = round((total - available) / total * 100, 1)
except: ram_pct = None
try:
    st = os.statvfs("/")
    disco_pct = round((1 - st.f_bavail / st.f_blocks) * 100, 1)
except: disco_pct = None
try:
    uptime_s = float(open("/proc/uptime").read().split()[0])
    h, m = divmod(int(uptime_s // 60), 60)
    uptime = f"{h}h {m}m"
except: uptime = None
try:
    gpu = float(re.search(r"temp=(\d.\d+)",
        subprocess.check_output(["vgencmd","measure_temp","core"]).decode()).group(1))
except: gpu = None
try:
    freq = int(re.search(r"frequency\((\d+)\)=(\d+)",
        subprocess.check_output(["vgencmd","measure_clock","arm"]).decode()).group(1)) // 1000000
except: freq = None
try:
    thr_hex = re.search(r"throttled=(0x[\da-fA-F]+)",
        subprocess.check_output(["vgencmd","get_throttled"]).decode()).group(1)
    thr_int = int(thr_hex, 16)
    throttle_ok = thr_int == 0
    throttle_flags = thr_hex
except: throttle_ok = True; throttle_flags = "0x0"
try:
    wifi_out = subprocess.check_output(["iwconfig","wlan0"], stderr=subprocess.DEVNULL).decode()

```

Bloque 3/3

```

wifi_dbm = int(re.search(r"Signal level=(-\d+)", wifi_out).group(1))
wifi_pct = max(0, min(100, 2*(wifi_dbm+100)))
except: wifi_dbm = None; wifi_pct = None
try:

```

Bloque de código 5. Tailscale, load average, filtro de outliers y construcción de series

Bloque 1/3

```

load = open("/proc/loadavg").read().split()
load1, load5, load15 = float(load[0]), float(load[1]), float(load[2])
except: load1=load5=load15=None
try:
    ip_ts = subprocess.check_output(["tailscale","ip","--4"], stderr=subprocess.DEVNULL).decode().strip()
except: ip_ts = None
return {"cpu_temp_c": temp, "voltaje_v": volt, "ram_uso_pct": ram_pct, "disco_uso_pct": disco_pct, "uptime":
    uptime, "gpu_temp_c": gpu, "freq_cpu_mhz": freq, "throttle_ok": throttle_ok, "throttle_flags":
    throttle_flags, "wifi_dbm": wifi_dbm, "wifi_pct": wifi_pct, "load1": load1, "load5": load5, "load15":
    load15, "tailscale_ip": ip_ts}

def generar_dashboard_json():
    os.makedirs("data/processed", exist_ok=True)
    nivel = leer_csv(RUTA_NIVEL)
    clima = leer_csv(RUTA_CLIMA)
    limite = 9999
    nivel_r = nivel[-limite:]

```

```

clima_r = clima[-limite:]

# Filtro outliers: no puede bajar menos de 5L (salvo vaciado) ni subir mas de 6L
nivel_filtrado = []
for i, row in enumerate(nivel_r):
    try:
        vol = float(row["volumen litros"])
    except:
        nivel_filtrado.append(row)
        continue
    if i == 0:
        nivel_filtrado.append(row)
        continue
    prev_vol = float(nivel_filtrado[-1]["volumen litros"])
    diff = vol - prev_vol
    if diff > 6: # pico exagerado hacia arriba
        row = dict(row)

```

Bloque 2/3

```

    row["volumen litros"] = str(prev_vol)
    elif diff < 0 and diff > -5: # baja poco = outlier por viento
        row = dict(row)
        row["volumen litros"] = str(prev_vol)
        nivel_filtrado.append(row)
nivel_r = nivel_filtrado
# Labels y series de nivel usan sus propios timestamps
labels_nivel = [f["timestamp"] for f in nivel_r]
nivel_l = [round(float(f["volumen litros"]), 2) for f in nivel_r if f.get("volumen litros")]

# Clima: submuestrear a 1 registro cada 4 (aprox 1 cada hora)
paso = max(1, len(clima_r) // 96)
clima_sub = clima_r[:paso]
labels = [f["timestamp"] for f in clima_sub if clima_sub else labels_nivel]
humedad = [round(float(f["humedad"]), 1) for f in clima_sub if f.get("humedad") and f["humedad"] != "N/A"]
temp_c = [f a c(f["temp exterior"]) for f in clima_sub if f.get("temp exterior") and f["temp exterior"] !=
"N/A"]
rocio_c = [f a c(f["punto rocio"]) for f in clima_sub if f.get("punto rocio") and f["punto rocio"] !=
"N/A"]
presion = [round(float(f["presion"]), 3) for f in clima_sub if f.get("presion") and f["presion"] != "N/A"]
viento = [mph_a_kmh(f["viento_vel"]) for f in clima_sub if f.get("viento_vel") and f["viento_vel"] !=
"N/A"]
dir_v = [round(float(f["viento_dir"]), 0) for f in clima_sub if f.get("viento_dir") and f["viento_dir"]
!= "N/A"]
lluvia = [round(float(f["lluvia_hora"]), 2) for f in clima_sub if f.get("lluvia_hora") and
f["lluvia_hora"] != "N/A"]
solar = [round(float(f["radiacion_solar"]), 1) for f in clima_sub if f.get("radiacion_solar") and
f["radiacion_solar"] != "N/A"]
uv = [round(float(f["uv"]), 1) for f in clima_sub if f.get("uv") and f["uv"] != "N/A"]
ultima_nivel = nivel[-1] if nivel else {}
ultima_clima = clima[-1] if clima else {}
kpis = {
    "nivel l": round(float(ultima_nivel.get("volumen litros", 0)), 2) if ultima_nivel else None,
    "humedad pct": round(float(ultima_clima.get("humedad", 0)), 1) if ultima_clima else None,

```

Bloque 3/3

```

    "temp_c": f a c(ultima_clima.get("temp exterior")) if ultima_clima else None,
    "punto rocio c": f a c(ultima_clima.get("punto rocio")) if ultima_clima else None,
    "presion inhg": round(float(ultima_clima.get("presion", 0)), 3) if ultima_clima else None,
    "viento_kmh": mph_a_kmh(ultima_clima.get("viento_vel")) if ultima_clima else None,
    "viento dir deg": round(float(ultima_clima.get("viento_dir", 0)), 0) if ultima_clima else None,
    "lluvia_hora_mm": round(float(ultima_clima.get("lluvia_hora", 0)), 2) if ultima_clima else None,
    "radiacion solar wm2": round(float(ultima_clima.get("radiacion solar", 0)), 1) if ultima_clima else None,
    "uv": round(float(ultima_clima.get("uv", 0)), 1) if ultima_clima else None
}
data = {

```

```
"actualizado": timestamp_ahora(),
```

Bloque de código 6. Estructura final dashboard_data.json con KPIs, sistema y series

```
"kpis": kpis,
"sistema": estado_sistema(),
"series": {
    "labels": labels,
    "labels_nivel": labels_nivel,
    "nivel_l": nivel_l,
    "humedad_pct": humedad,
    "temp_c": temp_c,
    "punto rocio c": rocio_c,
    "presion_inhg": presion,
    "viento_kmh": viento,
    "viento_dir_deg": dir_v,
    "lluvia_hora_mm": lluvia,
    "radiacion_solar_wm2": solar,
    "uv": uv
}
}
with open(RUTA_JSON, "w") as f:
    json.dump(data, f, indent=2)
print(f"[OK] dashboard_data.json generado: {timestamp_ahora()}")
return data

if __name__ == "__main__":
    data = generar_dashboard_json()
    print(f"KPIs: {data['kpis']}")
```

3.6 Scheduler, watchdog térmico y residencia operativa

scheduler.py unifica ejecución temporal y vigilancia de hardware. La rutina espera 600 s antes de la primera lectura para permitir levantamiento de red, inicia Flask en hilo daemon, ejecuta lectura y publicación cada 15 min, actualiza KPIs de la Pi cada 15 min y evalúa temperatura cada 5 min. El umbral crítico está fijado en 75 °C. Si se alcanza o supera, el nodo genera dashboard_data.json, sube el tablero y ejecuta apagado preventivo

Observación de consistencia: scheduler.py programa schedule.every(15).minutes.do(job_lectura), pero el mensaje impreso declara lecturas cada 60 min. El valor operativo real está en el código, no en el literal del log. Se debe corregir el mensaje y centralizar la frecuencia en settings.json

Bloque de código 7. Scheduler residente, hilo Flask, ciclos cada 15 min y watchdog a 75 °C

```
# Bloque 1/2
import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import schedule
import time
import threading
from scripts.main import tomar_lectura
from scripts.upload_github import subir_archivos, subir_dashboard
from scripts.process_data import generar_dashboard_json
from scripts.read_ws2000 import iniciar_servidor

TEMP_CRITICA = 75.0

def watchdog_termico():
    try:
```

```

with open("/sys/class/thermal/thermal_zone0/temp") as f:
    temp = int(f.read()) / 1000.0
print(f"[WATCHDOG] Temperatura CPU: {temp}°C")
if temp >= TEMP_CRITICA:
    print(f"[WATCHDOG] TEMPERATURA CRITICA {temp}°C - Generando reporte y apagando...")
    generar_dashboard_json()
    subir_dashboard()
    time.sleep(10)
    os.system("sudo shutdown -h now")
except Exception as e:
    print(f"[WATCHDOG] Error leyendo temperatura: {e}")

def job_lectura():
    print("[SCHEDULER] Tomando lectura del sensor...")
    tomar_lectura()
    print("[SCHEDULER] Generando y subiendo datos...")
    generar_dashboard_json()
    subir_archivos()

def job_sistema():
    print("[SCHEDULER] Actualizando KPIs del sistema Pi...")

```

Bloque 2/2

```

generar_dashboard_json()
subir_dashboard()

def iniciar_scheduler():
    print("[SCHEDULER] Esperando 10 min para que la red levante...")
    time.sleep(600)
    print("[SCHEDULER] Lectura inicial al arrancar...")
    job_lectura()
    schedule.every(15).minutes.do(job_lectura)
    schedule.every(15).minutes.do(job_sistema)
    schedule.every(5).minutes.do(watchdog_termico)
    print("[SCHEDULER] Activo - lecturas cada 60 min, KPIs Pi cada 15 min, watchdog cada 5 min")
    while True:
        schedule.run_pending()
        time.sleep(60)

if __name__ == "__main__":
    hilo_flask = threading.Thread(target=iniciar_servidor, daemon=True)
    hilo_flask.start()
    print("[SERVIDOR] Flask iniciado en puerto 7777")
    iniciar_scheduler()

```

3.7 Servicio systemd recomendado para ejecución persistente

El informe anterior documentó una unidad systemd con WorkingDirectory en /home/awa05/proyecto_agua y ExecStart hacia scripts/scheduler.py. La arquitectura actual debe conservar ese patrón, porque el scheduler depende de ruta relativa hacia data/raw y data/processed. El control de reinicio debe permanecer en Restart=always para recuperación tras caída del proceso

Bloque de código 8. Unidad systemd endurecida para esperar red y tailscaled

```

[Unit]
Description=AWA05 Telemetria Ambiental
After=network-online.target tailscaled.service
Wants=network-online.target tailscaled.service

[Service]
WorkingDirectory=/home/awa05/proyecto_agua
ExecStart=/home/awa05/proyecto_agua/venv/bin/python3 scripts/scheduler.py
Restart=always
RestartSec=10

```

```
User=awa05
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
```

3.8 Tailscale como plano privado de administración

El tablero operativo reporta `tailscale_ip=100.110.98.27`. Ese valor no proviene de configuración estática del JSON; se obtiene en `process_data.py` mediante la llamada `tailscale ip --4`. La métrica confirma que el nodo está unido a una tailnet o que al menos el binario Tailscale responde dentro del entorno de ejecución. En términos de operación, Tailscale agrega una red privada superpuesta para administración remota sin exponer directamente el puerto SSH a Internet público

Elemento Tailscale	Implementación observada o recomendada	Valor operativo
Extracción de IP	<code>subprocess.check_output(["tailscale", "ip", "--4"])</code>	100.110.98.27
Servicio base	<code>tailscaled.service</code>	Debe estar activo antes de <code>awa05.service</code>
Administración remota	SSH por IP 100.x.x.x o por nombre MagicDNS si está habilitado	Reduce dependencia de mDNS fuera de LAN
Diagnóstico	<code>tailscale status</code> , <code>tailscale ping</code> , <code>tailscale netcheck</code>	Verifica sesión, ruta DERP/directa, NAT y conectividad
Riesgo	Pérdida de autenticación o servicio detenido	<code>dashboard_data.json</code> reportaría None en <code>tailscale_ip</code>
Control recomendado	Agregar healthcheck explícito de <code>tailscaled</code> y registrar estado	Mejora trazabilidad de acceso remoto

Bloque de código 9. Integración Tailscale dentro de `estado_sistema()`

```
# Extracción actualmente implementada en process_data.py
try:
    ip_ts = subprocess.check_output(
        ["tailscale", "ip", "--4"],
        stderr=subprocess.DEVNULL
    ).decode().strip()
except:
    ip_ts = None

# Campo persistido dentro de dashboard_data.json
return {
    "cpu temp c": temp,
    "voltaje_v": volt,
    "ram uso_pct": ram_pct,
    "disco uso_pct": disco_pct,
    "uptime": uptime,
    "throttle ok": throttle_ok,
    "throttle_flags": throttle_flags,
    "wifi dbm": wifi_dbm,
    "wifi_pct": wifi_pct,
    "load1": load1,
    "load5": load5,
    "load15": load15,
    "tailscale_ip": ip_ts
}
```

Bloque de código 10. Rutina de diagnóstico Tailscale + servicio AWA05



```
# Comandos de verificación operativa en el nodo AWA05
```

```
sudo systemctl status tailscaled --no-pager
```

```
tailscale status
```

```
tailscale ip --4
```

```
tailscale netcheck
```

```
tailscale ping <equipo-administrador>
```

```
# Prueba de administración remota por red privada
```

```
ssh awa05@100.110.98.27
```

```
# Verificación conjunta con el servicio de telemetría
```

```
sudo systemctl status awa05.service --no-pager
```

```
journalctl -u awa05.service -n 120 --no-pager
```

```
journalctl -u tailscaled.service -n 120 --no-pager
```

Bloque de código 11. Dependencia systemd y campo de monitoreo Tailscale recomendado

```
# Endurecimiento recomendado si Tailscale debe estar disponible antes del scheduler
```

```
[Unit]
```

```
After=network-online.target tailscaled.service
```

```
Wants=network-online.target tailscaled.service
```

```
# Monitoreo mínimo recomendado dentro de dashboard_data.json
```

```
{  
  "tailscale_ip": "100.110.98.27",  
  "tailscale_status": "up",  
  "tailscale_last_check": "YYYY-MM-DD HH:MM:SS",  
  "tailscale_access_plane": "private-overlay"  
}
```

```
# Criterio de alerta operacional
```

```
# tailscale ip == None durante dos ciclos consecutivos => revisar tailscaled, autenticación y red
```

3.9 Sincronización con GitHub y separación de secretos

upload_github.py mantiene la regla correcta de no incrustar el token dentro del código. La función cargar_token() lee GITHUB_TOKEN desde .env y .gitignore excluye dicho archivo. La subida diferencia archivos crudos y dashboard_data.json. La estrategia create/update conserva continuidad de publicación en la rama main

Bloque de código 12. Publicación remota con PyGithub, token externo y actualización de dashboard

```
# Bloque 1/3
```

```
import os
```

```
import sys
```

```
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```
from github import Github, Auth
```

```
from scripts.utils import timestamp_ahora
```

```
def cargar_token():
```

```
    env_path = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), '.env')
```

```
    with open(env_path) as f:
```

```
        for linea in f:
```

```
            if linea.startswith('GITHUB_TOKEN'):
```

```
                return linea.strip().split('=')[1]
```

```
    raise ValueError('Token no encontrado en .env')
```

```
REPO_NAME = "geograficaaala/awa05-telemetria"
```

```
RAMA = "main"
```

```
ARCHIVOS = [
```

```
    "data/raw/nivel_raw.csv",
```

```
"data/raw/clima_raw.csv",
]

def subir_archivos():
    token = cargar_token()
    g = Github(auth=Auth.Token(token))
    repo = g.get_repo(REPO_NAME)
    for ruta_local in ARCHIVOS:
        if not os.path.exists(ruta_local):
            print(f"[SKIP] No existe: {ruta_local}")
            continue
        with open(ruta_local, "r") as f:
            contenido = f.read()
        ruta_github = ruta_local
        try:
            archivo = repo.get_contents(ruta_github, ref=RAMA)
            repo.update_file(
```

Bloque 2/3

```
        path = ruta_github,
        message = f"Actualizacion automatica {timestamp_ahora()}",
        content = contenido,
        sha = archivo.sha,
        branch = RAMA
    )
    print(f"[OK] Actualizado: {ruta_github}")
except Exception:
    repo.create_file(
        path = ruta_github,
        message = f"Creacion automatica {timestamp_ahora()}",
        content = contenido,
        branch = RAMA
    )
    print(f"[OK] Creado: {ruta_github}")
```

```
if __name__ == "__main__":
    subir_archivos()
```

```
def subir_dashboard():
    token = cargar_token()
    g = Github(auth=Auth.Token(token))
    repo = g.get_repo(REPO_NAME)
    ruta_local = "data/processed/dashboard_data.json"
    if not os.path.exists(ruta_local):
        print("[SKIP] No existe dashboard_data.json")
        return
    with open(ruta_local, "r") as f:
        contenido = f.read()
    try:
        archivo = repo.get_contents(ruta_local, ref=RAMA)
        repo.update_file(
            path = ruta_local,
            message = f"[sistema] KPIs Pi {timestamp_ahora()}",
            content = contenido,
            sha = archivo.sha,
```

Bloque 3/3

```
        branch = RAMA
    )
    print(f"[OK] dashboard_data.json actualizado")
except Exception:
    repo.create_file(
        path = ruta_local,
        message = f"[sistema] KPIs Pi {timestamp_ahora()}",
        content = contenido,
```



```
branch = RAMA
)
print(f"[OK] dashboard_data.json creado")
```

Bloque de código 13. Exclusiones de repositorio relevantes para secretos y datos generados

```
__pycache__ /
*.pyc
data/raw/peso_raw.csv
.env
data/raw/*.csv
data/processed/
scripts/ __pycache__ /
```

4. Trazabilidad de datos crudos y procesados

La cadena de datos conserva separación entre evidencia cruda y productos procesados. Esa separación es indispensable para auditoría, porque permite reconstruir KPIs, depurar filtros y repetir análisis sin sobrescribir el registro original. En el estado actual, nivel_raw.csv y clima_raw.csv son fuentes primarias; dashboard_data.json y statistical_analysis.json son derivados

Dataset	Registros o estructura	Inicio	Fin / actualización	Uso
nivel_raw.csv	1016	2026-04-23 12:29:11	2026-05-12 02:12:37	Distancia ultrasónica y volumen instantáneo
clima_raw.csv	496	2026-05-11 17:44:18	2026-05-12 02:12:38	Variables WS-2000 recibidas por HTTP
dashboard_data.json	kpis + sistema + series	2026-05-12 01:46:52	2026-05-12 01:46:52	Tablero operativo y salud de la Pi
statistical_analysis.json	B01	2026-05-08 23:42:37	2026-05-08 23:42:37	Inferencia estadística intra-bloque

4.1 Nivel crudo actual

El archivo nivel_raw.csv contiene 1016 registros. El intervalo mediano entre filas es 15,13 min, compatible con muestreo subhorario. El volumen medio del archivo actual es 9,85 L, con mínimo de 0,00 L y máximo de 20,02 L. La suma positiva cruda de deltas es 423,59 L en 336 incrementos detectados

Métrica	Valor
Registros totales	1016
Distancia media	52,93 cm
Distancia mínima / máxima	20,5 / 76,3 cm
Volumen medio	9,85 L
Volumen mínimo / máximo	0,00 / 20,02 L
Mediana de intervalo	15,13 min
Incrementos positivos crudos	336
Suma positiva cruda	423,59 L
Última lectura	2026-05-12 02:12:37; 10,04 L

4.2 Clima crudo reciente

El archivo clima_raw.csv contiene 496 registros recientes entre 2026-05-11 17:44:18 y 2026-05-12 02:12:38. Las conversiones usadas para tablero y análisis son °F a °C, mph a km/h y pulgadas/h a mm/h. La ventana reciente registra temperatura media de 20,87 °C, humedad media de 70,67 %, presión media de 24,995 inHg y viento medio de 0,31 km/h

Variable	Media	Mínimo	Mediana	Máximo	Unidad
Temperatura	20,87	18,78	21,00	21,50	°C
Humedad relativa	70,67	57,00	67,50	84,00	%
Punto de rocío	15,20	10,61	14,50	18,61	°C
Presión	24,995	24,950	24,997	25,027	inHg
Viento	0,31	0,00	0,00	5,41	km/h
Radiación solar	0,14	0,00	0,00	3,78	W/m ²

4.3 Consistencia temporal entre raw y análisis B01

El bloque estadístico B01 fue generado el 2026-05-08 y cubre nivel desde 2026-04-23 12:29:11 hasta 2026-05-06 09:27:12. El clima usado por B01 termina el 2026-04-29 06:13:50. En contraste, el archivo clima raw.csv del repositorio actual contiene registros de 2026-05-11 a 2026-05-12. Por tanto, la estadística B01 no debe leerse como análisis total de la carpeta raw actual. Debe leerse como bloque experimental versionado

5. Análisis estadístico del bloque B01

El bloque B01 formaliza la primera lectura inferencial del sistema. El diseño estadístico no usa la serie completa de nivel para correlacionar contra clima; limita la modelación al solapamiento efectivo entre nivel y meteorología. Esa decisión evita inducir correlaciones artificiales con periodos sin clima disponible. La unidad de análisis principal para modelos es la ventana horaria, con producción expresada en L/h

Concepto	Valor
Identificador	B01
Nombre	Bloque 01 · datos disponibles actuales
Periodo de nivel	2026-04-23 12:29:11 a 2026-05-06 09:27:12 · 12,8736 días
Periodo climático	2026-04-23 12:30:20 a 2026-04-29 06:13:50 · 5,7385 días
Solapamiento analítico	2026-04-23 12:30:20 a 2026-04-29 06:13:50 · 5,7385 días
Registros de agua	978
Registros climáticos válidos	8076
Ventanas horarias modeladas	100
Producción válida total	347,93 L
Producción solapada con clima	161,25 L
Producción por día de nivel	27,03 L/día

5.1 Control de calidad de datos B01

La base de nivel B01 contiene 978 registros. Antes de limpieza se detectaron 324 eventos positivos con suma de 413,50 L. Después de aplicar control físico, se retuvieron 318 eventos y se removieron 6 incrementos, todos clasificados como saltos positivos mayores a 6 L. La producción válida quedó en 347,93 L. La remoción equivale a 1,85 % de los eventos positivos crudos

En clima, el módulo recibió 8077 registros; 8076 conservaron timestamp válido y 1 registro quedó sin timestamp. Para inferencia producción-clima se usaron 100 ventanas con agua y clima y 130 eventos con clima más cercano. La decisión es conservadora porque no rellena periodos climáticos inexistentes

Control	Resultado	Interpretación
Registros de agua	978	Serie de base para derivar cambios positivos de volumen
Eventos positivos crudos	324	Incrementos antes de limpieza física
Eventos positivos válidos	318	Incrementos retenidos para análisis
Eventos removidos	6	Salto incompatibles con respuesta física ordinaria
Umbral de remoción	> 6 L	Regla de control de outliers positivos
Registros climáticos válidos	8076	Base meteorológica después de validar timestamp
Ventanas horarias modeladas	100	Base efectiva de modelos y correlaciones

5.2 Descripción estadística de producción

La producción por intervalo presenta $n=975$ observaciones, media de 0,3569 L, mediana de 0,00 L y desviación estándar de 0,843 L. La mediana cero y el cuartil superior de 0,05 L indican que la serie está dominada por intervalos sin incremento o con incremento muy pequeño. El percentil 95 alcanza 2,51 L y el máximo retenido llega a 3,92 L. La asimetría de 2,2423 y la curtosis de 3,4721 confirman distribución no simétrica, con cola positiva

Los eventos positivos válidos tienen $n=318$, media de 1,0941 L, mediana de 0,21 L e intervalo intercuartílico de 2,30 L. La media excede ampliamente la mediana, por lo que el rendimiento queda concentrado en eventos discretos de mayor aporte. La tasa por evento positivo presenta media de 4,3774 L/h y mediana de 0,8335 L/h, con coeficiente de variación de 107,92 %

La agregación horaria reduce ruido de medición y establece la base del modelo. En 100 ventanas horarias, la producción media es 1,6125 L/h, la mediana es 2,14 L/h, el IC95 % de la media va de 1,3095 a 1,9155 L/h y el máximo observado es 6,82 L/h. A escala diaria, $n=12$, media 28,9942 L/día, mediana 31,375 L/día e IC95 % entre 23,2304 y 34,7579 L/día

Variable	n	Media	Mediana	DE	P95	Máximo	IC95% media
Producción por intervalo L	975	0,3569	0,0000	0,8430	2,5100	3,9200	0,3039 - 0,4098
Delta positivo por evento L	318	1,0941	0,2100	1,1723	2,7775	3,9200	0,9648 - 1,2235
Producción horaria L/h	100	1,6125	2,1400	1,5268	3,5685	6,8200	1,3095 - 1,9155
Producción diaria L/día	12	28,9942	31,3750	9,0714	38,2590	38,8200	23,2304 - 34,7579

5.3 Descripción estadística de clima

El clima B01 se caracteriza por temperatura media de 20,4730 °C, humedad relativa media de 72,0222 %, punto de rocío medio de 15,1088 °C y VPD medio de 0,6996 kPa. La humedad relativa mantiene desviación estándar de 9,9663 puntos porcentuales y rango de 41,0 a 91,0 %. Estas magnitudes son coherentes con un entorno con variabilidad diaria suficiente para explorar respuesta de condensación, aunque no suficiente para atribución causal

La radiación solar tiene mediana de 0,4400 W/m² y media de 29,7752 W/m²; la diferencia indica presencia de muchas horas con radiación baja o nula y pocos valores altos. La lluvia horaria muestra mediana cero, p95 cero y máximo de 18,5928 mm/h, por lo que funciona más como indicador de eventos puntuales que como covariable continua estable

Variable	n	Media	Mediana	DE	Mín	Máx	Unidad
Temperatura	8076	20,4730	19,8889	2,7856	16,1111	27,0000	°C
Humedad relativa	8076	72,0222	72,0000	9,9663	41,0000	91,0000	%
Punto de rocío	8076	15,1088	15,6111	2,7690	4,8889	19,3889	°C
VPD	8076	0,6996	0,6547	0,3224	0,1964	1,9605	kPa
Humedad absoluta	8076	12,8273	13,0893	2,0485	6,5035	16,3470	g/m ³
Radiación solar	8076	29,7752	0,4400	57,4809	0,0000	789,0600	W/m ²

Variable	n	Media	Mediana	DE	Mín	Máx	Unidad
Viento	8076	0,5683	0,0000	1,2196	0,0000	8,6422	km/h
Lluvia	8076	0,0243	0,0000	0,5730	0,0000	18,5928	mm/h

5.4 Supuestos, normalidad y estacionariedad

Las pruebas de normalidad sobre producción horaria rechazan normalidad a $\alpha=0,05$. Este resultado obliga a leer media y desviación junto con mediana, percentiles y pruebas no paramétricas. La condición no invalida el análisis; define el tipo de inferencia admisible. Por eso el módulo reporta Shapiro-Wilk, D'Agostino-Pearson, Kruskal-Wallis, bootstrap y errores HAC

La estacionariedad de producción horaria es compatible con ADF y KPSS dentro del bloque, mientras que humedad presenta señal mixta: ADF la clasifica estacionaria y KPSS no. Esa diferencia es esperable en series ambientales con ciclos diarios y variación suave. El modelo lineal se mantiene como lectura exploratoria, no como predictor final de producción

Prueba	n	Estadístico	p-valor	Lectura
Shapiro-Wilk producción horaria	100	0.830533	0.0	No normal
D'Agostino producción horaria	100	14.192932	0.000828	No normal
ADF producción horaria	100	-10.308109	0.0	Compatible con estacionariedad
KPSS producción horaria	100	0.392119	0.080552	Compatible con estacionariedad
ADF humedad	100	-3.082973	0.027839	Compatible con estacionariedad
KPSS humedad	100	0.628408	0.020054	No estacionaria según KPSS

5.5 Producción por franja horaria y humedad

Franja	n	Media L/h	Mediana L/h	DE	IC95%
Madrugada	24	1,3004	1,2800	1,2461	0,7742 - 1,8266
Mañana	24	1,3471	0,3900	1,7170	0,6221 - 2,0721
Noche	24	2,2879	2,3800	1,4593	1,6717 - 2,9041
Tarde	28	1,5286	2,1200	1,5291	0,9357 - 2,1215

La franja Noche presenta la media más alta con 2,2879 L/h y mediana de 2,38 L/h. Madrugada y Mañana quedan cerca de 1,30 a 1,35 L/h, mientras Tarde alcanza 1,5286 L/h. Sin embargo, ANOVA clásico $p=0,087235$, Welch $p=0,080579$ y Kruskal-Wallis $p=0,138662$ no cruzan el umbral convencional de 0,05. La evidencia sugiere patrón operativo, pero no prueba diferencia estadística robusta entre franjas dentro de un bloque único

Tercil de humedad	n	Media L/h	Mediana L/h	DE	IC95%
Humedad alta	33	1,9891	2,3600	1,8483	1,3337 - 2,6445
Humedad baja	34	1,1456	0,4450	1,1897	0,7305 - 1,5607
Humedad media	33	1,7170	2,2500	1,3955	1,2222 - 2,2118

La lectura por terciles de humedad muestra gradiente ascendente desde humedad baja hacia humedad alta. La media aumenta desde 1,146 L/h hasta 1,989 L/h. El resultado es físicamente plausible para condensación, pero la prueba paramétrica se mantiene limítrofe y la prueba no paramétrica no confirma separación fuerte. En términos técnicos, el resultado debe formularse como hipótesis prioritaria para siguientes bloques

5.6 Correlaciones, controles parciales y rezagos

Variable	n	Pearson r	p Pearson	Spearman p	p Spearman	p FDR
rain_mm_h	100	-0,1171	0,24574	-0,1988	0,04739	0,33789
humidity_pct	100	0,2219	0,02646	0,1433	0,15496	0,29112

Variable	n	Pearson r	p Pearson	Spearman p	p Spearman	p FDR
abs_humidity_gm3	100	0,1317	0,19151	0,1372	0,17339	0,33789
dew_point_calc_c	100	0,1233	0,22165	0,1257	0,21273	0,33789
dew_point_c	100	0,1207	0,23178	0,1223	0,22544	0,33789
solar_wm2	100	-0,1329	0,18749	-0,1132	0,26231	0,33789
vpd_kpa	100	-0,1813	0,07109	-0,1045	0,30083	0,33789
wind_kmh	100	-0,0006	0,99547	-0,0709	0,48305	0,99547
uv	100	-0,1255	0,21342	-0,0703	0,48679	0,33789
pressure_hpa	100	0,1072	0,28831	0,0617	0,54212	0,35238
temp_c	100	-0,0842	0,40510	-0,0076	0,93990	0,44561

La correlación bivariada lineal más relevante es humedad relativa con $r=0,221947$ y $p=0,026465$. Después de corrección FDR, la señal queda como no concluyente. La relación monotónica más fuerte corresponde a lluvia horaria con $\rho=-0,198795$ y $p=0,047391$. La dirección negativa puede reflejar condiciones de evento, ruido temporal o estructura de bloque; no debe interpretarse como causalidad hidráulica

Variable	Controles	n	r parcial	p
humidity_pct	temp_c, solar_wm2	100	0,2066	0,03915
temp_c	humidity_pct, solar_wm2	100	0,1065	0,29174
vpd_kpa	temp_c, humidity_pct	100	0,0409	0,68634
dew_point_c	temp_c, solar_wm2	100	0,2036	0,04213
abs_humidity_gm3	temp_c, solar_wm2	100	0,2046	0,04117

Las correlaciones parciales retienen señales positivas para humedad relativa, punto de rocío y humedad absoluta al controlar por temperatura y radiación solar. Las magnitudes permanecen bajas, alrededor de r parcial 0,20. En el contexto de ingeniería del sistema, esto no se usa para predicción directa; se usa para priorizar hipótesis físicas y definir variables del siguiente bloque experimental

5.7 Modelo OLS/HAC y diagnóstico de residuos

El modelo OLS con errores HAC utiliza humidity_pct, temp_c, vpd_kpa, solar_wm2, wind_kmh como covariables y $n=100$ ventanas. El R^2 es 0,068129, el R^2 ajustado es 0,018561, el MAE es 1,2244 L/h y el RMSE es 1,4665 L/h. Estos valores indican capacidad explicativa baja. La utilidad del modelo es diagnóstica: permite evaluar dirección de coeficientes, influencia y residuos, no producir pronósticos confiables

Indicador	Valor
Variables	humidity_pct, temp_c, vpd_kpa, solar_wm2, wind_kmh
n	100
R^2	0,068129
R^2 ajustado	0,018561
MAE	1,2244 L/h
RMSE	1,4665 L/h
Durbin-Watson	2,1705
Ljung-Box p	0,39564
Breusch-Pagan p	0,29840

Término	Coficiente	EE HAC	t HAC	p	IC95% HAC
const	-6,22928	6,37840	-0,9766	0,33126	-18,8938 - 6,4352

Término	Coefficiente	EE HAC	t HAC	p	IC95% HAC
humidity_pct	0,09264	0,10665	0,8686	0,38728	-0,1191 - 0,3044
temp_c	-0,00998	0,24476	-0,0408	0,96755	-0,4960 - 0,4760
vpd_kpa	1,79685	3,76464	0,4773	0,63426	-5,6779 - 9,2716
solar_wm2	-0,00377	0,00455	-0,8280	0,40976	-0,0128 - 0,0053
wind_kmh	0,16747	0,15638	1,0709	0,28696	-0,1430 - 0,4780

Ningún coeficiente individual conserva significancia bajo errores HAC. La lectura correcta no es descartar el mecanismo físico, sino reconocer que el bloque B01 no contiene información suficiente para estimar efectos independientes de variables altamente correlacionadas. El diagnóstico de autocorrelación de residuos no es crítico según Ljung-Box $p=0,395643$, pero la normalidad residual vuelve a rechazarse

5.8 Modelos predictivos y multicolinealidad

Modelo	R ² interno	MAE interno	CV R ² medio	CV MAE	CV temporal MAE
ridge_cv	0,1019	1,2120	-0,6135	1,3842	1,2480
lasso_cv	0,0000	1,2953	-0,1754	1,3316	1,2146
random_forest	0,5593	0,8369	-0,3788	1,4130	1,3051
gradient_boosting	0,5426	0,8971	-0,3266	1,3952	1,3338

Ridge, Lasso, Random Forest y Gradient Boosting presentan R² de validación cruzada negativo. Random Forest y Gradient Boosting ajustan mejor dentro de muestra, pero esa mejora no se sostiene fuera de muestra. El mejor MAE de validación cruzada simple corresponde a lasso_cv con 1,331614 L/h, aunque su R² interno es 0,0 por regularización completa. El resultado técnico es que el bloque B01 todavía no soporta pronóstico operacional confiable

Variable	VIF	Estado
temp_c	1.490,6137	alto
dew_point_c	865,7352	alto
abs_humidity_gm3	602,2909	alto
vpd_kpa	451,8441	alto
humidity_pct	174,7480	alto
solar_wm2	20,8180	alto
uv	11,1044	alto
wind_kmh	4,8206	ok
pressure_hpa	1,5466	ok
rain_mm_h	1,0943	ok

La multicolinealidad es severa. Temperatura, punto de rocío, humedad absoluta, VPD y humedad relativa no deben interpretarse simultáneamente como predictores independientes dentro de OLS. Para siguientes bloques se recomienda seleccionar variables físicas no redundantes o aplicar reducción dimensional antes de emitir coeficientes interpretables

5.9 Bootstrap e incertidumbre

Magnitud	n boot	IC95% media	IC95% mediana
Producción horaria	3000	1,3320 - 1,9164 L/h	0,5200 - 2,3750 L/h
Tasa de evento positivo	3000	3,8758 - 4,8879 L/h	0,8299 - 1,4289 L/h
Producción diaria	3000	23,6563 - 33,4692 L/día	23,5000 - 35,4150 L/día

El bootstrap con 3.000 repeticiones proporciona intervalos no paramétricos robustos frente a no normalidad. La media de producción horaria queda entre 1,33199 y 1,91641 L/h, mientras la producción diaria media queda entre 23,65635 y 33,46917 L/día. Estos intervalos son más informativos que un único promedio porque expresan incertidumbre de bloque

5.10 Síntesis estadística operacional

- El periodo crudo de producción/nivel no es exactamente 23–30: inicia 2026-04-23 12:29:11 y termina 2026-05-06 09:27:12. El clima inicia 2026-04-23 12:30:20 y termina 2026-04-29 06:13:50.
- Para inferencia producción-clima, el solapamiento real termina el 2026-04-29 06:13:50; por tanto el bloque actual funciona como línea base, no como comparación inter-bloques.
- La asociación monotónica más fuerte en ventanas horarias fue rain_mm_h con ρ de Spearman = -0.198795; debe interpretarse como asociación intra-bloque, no causalidad.
- El mejor desempeño predictivo por MAE de validación cruzada fue lasso_cv con $MAE \approx 1.331614$ L/h; el resultado está limitado por tamaño muestral y autocorrelación temporal.
- ANOVA/post hoc se reporta como análisis intra-bloque por escenarios horarios/climáticos; ANOVA inter-bloques se debe activar cuando existan al menos 3 bloques y preferiblemente 5+.

Limitaciones estadísticas declaradas:

- Solo existe un bloque experimental; por rigor, la inferencia inter-bloques está deshabilitada.
- El periodo climático termina antes que el periodo de producción, por lo que los modelos usan únicamente el solapamiento real entre agua y clima.
- La producción se deriva de cambios positivos de volumen y no de un caudalímetro dedicado; se aplican reglas de limpieza para saltos físicamente anómalos.
- Las observaciones horarias pueden presentar autocorrelación; por eso el OLS se reporta con errores robustos HAC/Newey-West.
- Las asociaciones estadísticas no implican causalidad; se interpretan como hipótesis para los siguientes bloques.

6. Comparativo de calidad de agua - Lote 1

El módulo de calidad de agua estructura el Lote 1 de AWA05 contra fuentes comerciales y locales. La matriz no debe confundirse con ensayo pareado entre productos. AWA05 cuenta con evidencia de laboratorio Atitlab sobre muestra específica; las fuentes comerciales se comparan con valores publicados o información disponible. La utilidad del bloque es técnica y documental: ubicar el perfil fisicoquímico y microbiológico de AWA05 dentro de una matriz común

Categoría	Parámetros
Aniones	2
Desinfección	2
Fisicoquímico	5
Metales	5
Microbiología	7
Minerales	6
Nutrientes	2
Operativo	1

6.1 Resultados AWA05 Lote 1

El Lote 1 presenta pH 7,22, TDS 20,2 mg/L, conductividad 14,4 μ S/cm, turbidez 0,85 NTU, color real 0 UPtCo, dureza 25 mg/L CaCO₃ y alcalinidad 40 mg/L CaCO₃. Ese perfil corresponde a agua de baja mineralización y baja turbidez. La matriz marca cumplimiento para los parámetros con referencia declarada, con nota específica para calcio por inconsistencia de unidad en el documento fuente

Parámetro	Resultado AWA05	Referencia	Estado	Nota
pH	7.22 pH	6.5 - 8.5 pH	cumple	Dentro de 6.5–8.5

Parámetro	Resultado AWA05	Referencia	Estado	Nota
Sólidos totales disueltos	20.2 mg/L	— - 1000 mg/L	cumple	Muy baja mineralización
Conductividad eléctrica	14.4 µS/cm	— - 1500 µS/cm	cumple	Muy baja conductividad
Turbidez	0.85 NTU	— - 15 NTU	cumple	Muy bajo frente al LMA
Color real	0 UPtCo	— - 35 UPtCo / ACU	cumple	Sin color real reportado
Temperatura	23.1 °C	15 - 34 °C	cumple	
Dureza total	25 mg/L CaCO ₃	— - 500 mg/L CaCO ₃	cumple	
Alcalinidad total	40 mg/L CaCO ₃	30 - 400 mg/L CaCO ₃	cumple	
Calcio	100† mg/L	— - 150 mg/L	advertencia	La unidad del PDF aparece inconsistente; revisar contra laboratorio antes de conclusión fuerte.
Magnesio	35.50 mg/L	— - 100 mg/L	cumple	
Cloruros	62 mg/L	— - 250 mg/L	cumple	
Sulfatos	1.00 mg/L	— - 250 mg/L	cumple	
Nitratos	0.02 mg/L	— - 50 mg/L	cumple	
Nitritos	0.006 mg/L	— - 3 mg/L	cumple	

La fracción microbiológica reporta <2 UFC/mL o <2 NMP/mL en los indicadores cargados: recuento en placa, coliformes totales, coliformes fecales, E. coli, Salmonella spp, Vibrio cholerae y Enterobacter cloacae. El resultado es favorable para el lote analizado, pero debe mantenerse acotado al lote y a la fecha de muestreo

Parámetro	Resultado AWA05	Referencia	Estado
Arsénico	0.0 mg/L	≤ 0.01 mg/L	cumple
Cobre	0.021 mg/L	≤ 1.5 mg/L	cumple
Cromo	0.014 mg/L	≤ 1.5 mg/L	cumple
Manganeso	0.10 mg/L	≤ 0.4 mg/L	advertencia
Hierro	0.01 mg/L	≤ 1 mg/L	cumple
Cloro total	0.0 mg/L	≤ 1 mg/L	cumple
Cloro residual libre	0.0 mg/L	≤ 1 mg/L	cumple
Recuento aeróbico en placa	<2 UFC/mL	≤ 500 UFC/mL	cumple
Coliformes totales	<2 NMP/mL	≤ 2 NMP	cumple
Coliformes fecales	<2 NMP/mL	≤ 2 NMP	cumple
Escherichia coli	<2 NMP/mL	≤ 2 NMP	cumple
Salmonella spp	<2 NMP/mL	≤ 2 NMP/mL	cumple
Vibrio cholerae	<2 NMP/mL	≤ 2 NMP/mL	cumple
Enterobacter cloacae	<2 NMP/mL	≤ 2 NMP/mL	cumple

6.2 Comparativo con fuentes comerciales y locales

Fuente	Evidencia	pH	TDS	Conductividad	Cobertura	Score interno
AWA05 Lote 1	laboratorio	7.22 pH	20.2 mg/L	14.4 µS/cm	28	99
VOSS Still	publico	6.2 pH	26 mg/L	38 µS/cm	9	93
Acqua Panna	publico	7.66 pH	138 mg/L	237 µS/cm	13	98
FIJI Water	publico	7.8 pH	230 mg/L	290 µS/cm	12	100

Fuente	Evidencia	pH	TDS	Conductividad	Cobertura	Score interno
Essentia	publico	9.8 pH	58.3 mg/L	111 μ S/cm	18	97
S.Pellegrino	publico	5.7 pH	900 mg/L	1200 μ S/cm	16	77
Evian	publico	7.2 pH	345 mg/L	—	9	97
Ecofiltro	publico parcial	—	—	—	3	100
Salvavidas	institucional	—	—	—	0	—
Smartwater	publico parcial	—	—	—	9	100

AWA05 presenta TDS de 20,2 mg/L, comparable con aguas de baja mineralización y menor que varias fuentes comerciales minerales. La conductividad de 14,4 μ S/cm refuerza esa clasificación. El score interno de matriz no es dictamen legal independiente; solo resume ponderación técnica dentro del tablero. Las celdas vacías en marcas comerciales se interpretan como dato no cargado o no publicado, no como incumplimiento

La comparación regulatoria estricta exigiría certificados homogéneos por lote, método, fecha, laboratorio, unidad y límite de cuantificación. Mientras esa condición no exista, el benchmark debe usarse como contexto técnico y no como afirmación de superioridad sanitaria frente a marcas comerciales

7. Integración operativa entre nodo, estadística y calidad

El avance más relevante del sistema es la separación de responsabilidades. La Pi asegura telemetría y estado operativo; el pipeline de datos asegura trazabilidad; el módulo estadístico evalúa rendimiento; el módulo de calidad evalúa composición del agua. Esa separación reduce errores de interpretación: producir más agua no implica automáticamente mejor calidad, y cumplir un lote de calidad no implica estabilidad productiva futura

Dominio operativo

Estado de Pi, Tailscale, WiFi, CPU, RAM, disco, watchdog y publicación

Dominio hídrico

Distancia ultrasónica, volumen instantáneo, deltas positivos, producción por intervalo, L/h y L/día

Dominio meteorológico

Humedad, temperatura, punto de rocío, VPD, radiación, viento, presión y lluvia

Dominio sanitario

Parámetros fisicoquímicos, metales, desinfección, microbiología y benchmark de fuentes

Regla técnica

Cada dominio se evalúa con su evidencia propia y se integra solo después de validar periodo, unidad y método

7.1 Riesgos técnicos abiertos

Riesgo	Evidencia	Acción técnica
Desalineación de frecuencia	settings.json declara 60 min; scheduler ejecuta 15 min y el log dice 60 min	Centralizar frecuencia en settings.json y eliminar literales divergentes
Estadística no sincronizada con raw reciente	B01 no incluye clima_raw.csv del 11-12 mayo	Generar bloques versionados B02, B03 y manifiesto de fechas
Outliers de volumen	B01 removió 6 saltos positivos >6 L	Publicar dataset limpio junto a raw y documentar regla en README
Flask dev server	read_ws2000.py usa app.run host 0.0.0.0 port 7777	Migrar a WSGI o aislar por red si aumenta criticidad
Tailscale no supervisado	Solo se registra IP, no estado completo de tailnet	Agregar tailscale status, estado de tailscaled y última verificación
Unidad calcio	Valor 100† mg/L marcado como inconsistente	Solicitar confirmación formal al laboratorio
Comparabilidad comercial	Marcas no comparten lote ni método	Mantener benchmark como informativo hasta

Riesgo	Evidencia	Acción técnica
		tener certificados homogéneos

8. Conclusiones técnicas

- a) El nodo AWA05 alcanzó estado operativo de telemetría embebida: registra 1.016 filas de nivel, 496 filas recientes de clima, publica dashboard_data.json y reporta salud de Raspberry Pi con CPU 35,5 °C, RAM 10,8 %, disco 10,4 %, WiFi -66 dBm, throttling 0x0 y Tailscale IPv4 100.110.98.27
- b) La arquitectura informática ya contiene adquisición GPIO, ingesta HTTP, persistencia CSV, procesamiento JSON, publicación GitHub, watchdog térmico y plano privado Tailscale; el sistema dejó de ser una prueba aislada y opera como nodo de borde con trazabilidad de datos y estado
- c) El driver ultrasónico fue fortalecido con 11 muestras, mediana, timeout de 0,04 s, saturación geométrica y validación de rango; con ALTURA_TOTAL 72,5 cm, ALTURA_MAX_AGUA 38,3 cm y RADIO 12,9 cm, el volumen máximo teórico queda en 20,02 L
- d) El bloque estadístico B01 contiene 978 registros de agua, 8.076 registros climáticos válidos, 318 eventos positivos retenidos y 100 ventanas horarias modeladas; la producción válida acumulada es 347,93 L y la producción media diaria es 28,9942 L/día
- e) La producción horaria de B01 tiene media 1,6125 L/h, mediana 2,14 L/h e IC95 % de 1,3095 a 1,9155 L/h; la distribución no es normal, por lo que la interpretación correcta requiere percentiles, pruebas no paramétricas, bootstrap y errores robustos
- f) La evidencia climática sugiere relación positiva débil entre humedad y producción, con Pearson $r=0,221947$ para humedad relativa y r parcial cercano a 0,20 tras controlar temperatura y radiación; el resultado es compatible con hipótesis de condensación pero no prueba causalidad
- g) El modelo multivariable OLS/HAC presenta $R^2=0,068129$, R^2 ajustado=0,018561, MAE=1,2244 L/h y ausencia de coeficientes significativos bajo errores robustos; su valor actual es diagnóstico y no predictivo
- h) La comparación de calidad de agua del Lote 1 ubica a AWA05 como agua de baja mineralización, con pH 7,22, TDS 20,2 mg/L, conductividad 14,4 μ S/cm, turbidez 0,85 NTU y microbiología reportada como <2 en indicadores críticos
- i) La conclusión sanitaria queda limitada al Lote 1 y a la muestra del 3 de mayo de 2026; para control de calidad estadístico se requieren lotes adicionales, unidades verificadas, repetición temporal y certificados homogéneos
- j) La siguiente etapa debe cerrar cuatro brechas técnicas: unificar frecuencia de muestreo, versionar bloques estadísticos, ampliar monitoreo Tailscale/tailscaled y publicar dataset procesado limpio con reglas explícitas de exclusión

9. Pendientes técnicos de siguiente iteración

Área	Pendiente técnico	Criterio de cierre
Configuración	Unificar frecuencia entre settings.json, scheduler.py, dashboard y logs	Un solo parámetro gobierna ciclo de lectura y mensajes operativos
Tailscale	Registrar estado de tailscaled, tailnet, MagicDNS y última verificación	dashboard_data.json reporta IP, estado y timestamp de verificación
Estadística	Regenerar análisis con raw reciente y abrir bloques B02/B03	HTML estadístico lista bloques versionados y fechas de solapamiento
Datos limpios	Publicar eventos válidos, ventanas horarias y dataset filtrado	data/processed contiene archivos limpios auditables
Calidad	Confirmar unidad del calcio con Atitlab	Matriz conserva valor sin marca de inconsistencia o anexa aclaración formal
Laboratorio	Repetir calidad de agua en al menos tres lotes adicionales	Reporte de media, dispersión y cumplimiento por lote
Despliegue	Evaluar migración de Flask a WSGI supervisado	Servicio HTTP con logs y reinicio controlado
Operación	Agregar logrotate, alertas y retención de bitácora	Logs persistentes sin crecimiento indefinido

Anexo A. Configuración declarativa vigente

Bloque de código 14. settings.json actual

```
{
  "sitio": "AWA05",
  "frecuencia_muestreo_minutos": 60,
  "zona_horaria": "America/Guatemala",
  "rutas": {
    "datos_raw": "data/raw/",
    "datos_procesados": "data/processed/",
    "backups": "data/backups/",
    "logs": "logs/"
  },
  "github": {
    "repo": "",
    "rama": "main"
  }
}
```

Bloque de código 15. calibration.json residual de etapa de celda de carga

```
{
  "factor_calibracion": 1.0,
  "offset_tara": 0,
  "unidades": "kg",
  "limite_minimo": 0,
  "limite_maximo": 20
}
```

Anexo B. Ecuaciones estadísticas documentadas en el módulo B01

ID	Descripción	Expresión
block_production	Producción total del bloque	$Y_b = V_{b,final} - V_{b,inicial}$
event_production	Producción por intervalo	$Y_t = \max(V_t - V_{t-1}, 0)$
rate	Tasa horaria	$P_t = \frac{V_t - V_{t-1}}{\Delta t}$
mean	Media muestral	$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
variance	Varianza muestral	$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$
vpd	Déficit de presión de vapor	$VPD = e_s(T) - e_a = e_s(T) \left(1 - \frac{RH}{100}\right)$
es	Presión de vapor de saturación	$e_s(T) = 0.6108 \exp\left(\frac{17.27T}{T+237.3}\right)$
anova	Estadístico F de ANOVA	$F = \frac{MS_{between}}{MS_{within}}$
kruskal	Kruskal-Wallis	$H = \frac{12}{N(N+1)} \sum_{i=1}^k n_i (\bar{R}_i - \bar{R})^2$
vif	Factor de inflación de varianza	$VIF_j = \frac{1}{1 - R_j^2}$
ols	Modelo lineal multivariado	$Y_t = \beta_0 + \beta_1 RH_t + \beta_2 T_t + \beta_3 VPD_t + \beta_4 S_t + \beta_5 W_t + \epsilon_t$
r2	Coefficiente de determinación	$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$
mae	Error absoluto medio	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $
rmse	Raíz del error cuadrático medio	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$
dw	Durbin-Watson	$DW = \frac{\sum_{t=2}^n (e_t - e_{t-1})^2}{\sum_{t=1}^n e_t^2}$
cook	Distancia de Cook	$D_i = \frac{e_i^2}{p \cdot MSE} \frac{h_{ii}}{(1 - h_{ii})^2}$

